Rapport

Deep Learning

du perceptron au RNN

Karim Ryahi

Formation Data Engineer,

Simplon

TP Introduction au Deep Learning

1. Synthèse du travail réalisé

Ce rapport couvre les six exercices du TP, avec une attention particulière à l'exercice 3 (CNN sur MNIST) dont le code est expliqué étape par étape. Règle suivie partout : on sépare d'abord train et test, puis on calcule les statistiques (moyenne, médiane, scaler) sur le train uniquement, pour éviter toute fuite de données.

Exercice	Modèle	Résultat obtenu
1. Perceptron	1 neurone, sigmoïde, descente de gradient (AND, puis NOR)	Entropie croisée : perte 0,0194 après 1800 époques, sorties finales [0 ; 0,02 ; 0,02 ; 0,972]
2. Iris	MLP 4-8-3 (ReLU, softmax)	96,67 % de précision sur le test, 67 paramètres
2 (suite). Titanic	MLP dense, sigmoïde, EarlyStopping	Classification binaire de la survie ; le corrigé du TP donne 81,56 %
3. MNIST	CNN à 3 Conv2D	98,82 % sur le test, 130 890 paramètres
4. Fashion-MNIST	Même CNN	90,95 % sur le test
6. Victor Hugo	Embedding + SimpleRNN, puis LSTM	Perte finale 1,5488 (SimpleRNN) contre 0,4322 (LSTM)

2. Exercice 1 : le perceptron

Un neurone calcule $z = w1 \cdot x1 + w2 \cdot x2 + b$, puis applique la sigmoïde pour obtenir une valeur entre 0 et 1. Les poids sont corrigés par descente de gradient : $w \leftarrow w - \eta \cdot \text{gradient}$.

Deux pertes ont été comparées. Avec l'erreur quadratique (MSE) et $\eta = 0{,}1$, la perte passe de 0,367 à 0,035 en 2000 époques : le neurone apprend, mais lentement, car le gradient est multiplié par $\hat{y}(1 - \hat{y})$, très petit quand la sortie est loin de 0,5. Avec l'entropie croisée et $\eta = 0{,}5$, le gradient se simplifie en $(\hat{y} - y)$ et la convergence est bien plus nette. Un neurone seul ne sait apprendre que des fonctions linéairement séparables (AND, OR, NOR) ; XOR demande plusieurs couches.

3. Exercice 2 : MLP sur Iris et Titanic

Iris. Les 150 fleurs ont 4 mesures et 3 classes. Étapes du notebook : split stratifié 80/20, StandardScaler ajusté sur le train, puis un MLP Dense(8, ReLU) → Dense(3, softmax). Le nombre de paramètres se vérifie à la main : $4 \times 8 + 8 = 40$, puis $8 \times 3 + 3 = 27$, soit 67. La perte sparse_categorical_crossentropy est utilisée car les étiquettes sont des entiers 0, 1, 2 (avec un encodage one-hot, il faudrait categorical_crossentropy). Entraînement : Adam, 100 époques, batch 5, 10 % du train en validation. Résultat : 96,67 % sur le test.

Titanic. Même démarche, avec en plus : médiane de l'âge et mode de l'embarquement calculés sur le train pour remplacer les valeurs manquantes, get_dummies pour les variables catégorielles (colonnes du test alignées sur celles du train), standardisation de l'âge et du tarif. Sortie : 1 neurone sigmoïde et binary_crossentropy. EarlyStopping (patience 10, restore_best_weights) arrête l'entraînement quand la perte de validation ne baisse plus.

4. Exercice 3 : CNN sur MNIST

4.1 Rôle de chaque couche

Couche	Rôle	Paramètres
Conv2D	Des filtres 3×3 glissent sur l'image et détectent des motifs (bords, coins, courbes). Chaque filtre produit une carte de caractéristiques.	Conv2D(32) : 320 ; Conv2D(64) : 18 496 ; Conv2D(64) : 36 928
MaxPooling2D	Garde le maximum de chaque bloc 2×2 : taille divisée par 2, tolérance aux petits décalages.	0
Flatten	Met les 3×3×64 cartes à plat : 576 valeurs.	0
Dense	Combine toutes les caractéristiques pour décider. Dense(128) puis Dense(10, softmax) qui donne une probabilité par chiffre.	73 856 puis 1 290
Dropout(0,5)	Éteint au hasard la moitié des neurones à chaque pas d'entraînement pour limiter le surapprentissage. Inactif au test.	0

Tailles successives : 28×28×1 → 26×26×32 → 13×13×32 → 11×11×64 → 5×5×64 → 3×3×64 → 576 → 128 → 10. Total : 130 890 paramètres.

4.2 Le code, étape par étape

Prétraitement. Les images MNIST (60 000 en entraînement, 10 000 en test) ont des pixels de 0 à 255. On les convertit en float32 et on divise par 255 pour obtenir des valeurs entre 0 et 1, ce qui stabilise et accélère l'apprentissage.

np.expand_dims ajoute l'axe du canal : (N, 28, 28) devient (N, 28, 28, 1), format attendu par Conv2D.

Architecture. Trois Conv2D (32, 64, 64 filtres) séparées par deux MaxPooling2D, puis Flatten, Dense(128, ReLU), Dropout(0,5) et Dense(10, softmax). Les couches de convolution extraient des motifs de plus en plus abstraits ; la partie dense classe.

Compilation et entraînement. Optimiseur Adam (η = 0,001 par défaut), perte sparse_categorical_crossentropy car les étiquettes sont des entiers de 0 à 9, métrique accuracy. fit avec 5 époques, batch de 64 et validation_split = 0,1 : la validation est prélevée dans le train, le test reste inconnu jusqu'à l'évaluation finale.

Évaluation. model.evaluate sur le test donne 98,82 %. La matrice de confusion (heatmap) montre où le modèle se trompe, et classification_report détaille précision, rappel et F1 par chiffre : tous entre 0,95 et 1,00, le chiffre 5 ayant le rappel le plus faible (0,95). La précision de validation reste stable autour de 98,6 % dès la première époque : pas de surapprentissage visible.

5. Exercice 4 : Fashion-MNIST

Les données ont le même format que MNIST (28×28, 10 classes), donc le même pipeline et le même réseau sont réutilisés, avec 10 époques. Résultat : 90,95 % sur le test, soit près de 8 points de moins que MNIST : la tâche est plus difficile, pas le réseau moins bon.

Classe	F1-score
Pantalon	0,99
Sandale	0,98
Sac, Bottine	0,97
Sneaker	0,96

Robe	0,91
Pull	0,87
Manteau	0,86
T-shirt	0,85
Chemise	0,73

La Chemise est la classe la plus difficile (F1 0,73), suivie des hauts T-shirt, Manteau et Pull. À confirmer sur ta matrice de confusion : regarde avec quelles classes la Chemise est le plus souvent confondue.

6. Exercice 6 : RNN et génération de texte (réponses aux 10 questions)

1. **Tokenizer.** Il construit un dictionnaire mot → entier (selon la fréquence) et transforme chaque texte en suite d'entiers. Un réseau ne manipule que des nombres ; num_words = 1000 garde les 1000 mots les plus fréquents.
2. **pad_sequences.** Au début d'une ligne, la fenêtre contient moins de 3 mots. Le padding (zéros devant) donne à tous les exemples la même longueur 3. Sans lui, on ne peut pas former un tableau numpy rectangulaire et le modèle refuse les entrées.
3. **vocab_size et embedding_dim.** vocab_size = 1000 est le nombre de mots connus (entrées de l'Embedding et nombre de sorties du softmax). embedding_dim = 16 est la taille du vecteur qui représente chaque mot.
4. **Embedding.** Table apprise qui associe à chaque indice de mot un vecteur dense de 16 valeurs. Plus compact qu'un one-hot de 1000 valeurs, et des mots proches par le sens ont des vecteurs proches. On le place avant le RNN car celui-ci attend des vecteurs de nombres réels, pas des indices.
5. **SimpleRNN et LSTM.** Le Simple RNN a un état caché simple et oublie vite le début de la séquence (gradient qui s'évanouit). Le LSTM ajoute une cellule mémoire et des portes (entrée, oubli, sortie) qui décident quoi garder. On préfère le LSTM pour les dépendances longues. Ici : perte

finale 1,5488 (Simple RNN, précision ≈ 65 %) contre 0,4322 (LSTM, précision ≈ 88 %).

6. **window_size = 3.** Le modèle prédit le mot suivant à partir des 3 mots précédents, ce qui donne 11 541 exemples. Une fenêtre courte limite le contexte, donc la cohérence du texte ; une fenêtre plus longue apporte du contexte mais plus de calcul et plus de padding.
7. **sparse_categorical_crossentropy.** La sortie est un softmax sur 1000 classes (les mots) et la cible est l'indice entier du mot. La version « sparse » évite de construire une matrice one-hot de 11 541 × 1000.
8. **EarlyStopping.** Avec monitor = loss, patience = 10 et restore_best_weights, l'entraînement s'arrête quand la perte ne baisse plus pendant 10 époques et on récupère les meilleurs poids. Utile pour ne pas faire les 500 époques pour rien : le LSTM s'est arrêté vers l'époque 424.
9. **Courbe de perte.** Le modèle converge bien quand la courbe baisse puis forme un plateau. Le SimpleRNN plafonne vers 1,55 alors que le LSTM descend à 0,43. Attention : c'est la perte d'entraînement, sans jeu de validation. Elle ne prouve pas que le modèle généralise, et un 0,43 si bas peut traduire de la mémorisation sur un petit corpus.
10. **Texte généré.** Il ressemble à de la poésie (mots, rythme) mais n'est ni grammatical ni cohérent : contexte de 3 mots seulement, petit corpus, vocabulaire limité à 1000 mots (les autres sont ignorés), aucune notion de phrase. La température règle le hasard : 0,5 donne un texte plus répétitif, 1,2 un texte plus varié mais plus désordonné.

7. Conclusion

Le parcours va d'un neurone unique à des réseaux convolutifs et récurrents. Trois idées reviennent : séparer les données avant tout calcul de statistiques, choisir la perte adaptée à la sortie (sigmoïde avec entropie croisée binaire, softmax avec entropie croisée catégorielle), et surveiller la validation pour détecter le surapprentissage. Le même CNN obtient 98,82 % sur MNIST et 90,95 % sur Fashion-MNIST : la difficulté vient des données. Le LSTM bat nettement le SimpleRNN sur le texte, mais un modèle à fenêtre de 3 mots reste loin d'écrire comme Victor Hugo.

1. Que fait le Tokenizer et pourquoi est-il nécessaire ?

Le Tokenizer transforme les mots en indices numériques. Il construit un dictionnaire mot → indice basé sur les mots les plus fréquents du corpus. C'est nécessaire car un réseau de neurones ne comprend que des nombres, pas du texte brut.

2. Pourquoi utilise-t-on pad_sequences ?

Pour que toutes les séquences aient la même longueur. Un RNN attend des entrées de taille fixe. Sans pad_sequences, on ne pourrait pas traiter les séquences de longueurs différentes (certaines font 1 mot, d'autres 3). On complète avec des 0 au début (padding='pre').

3. Quelle est la différence entre vocab_size et embedding_dim ?

vocab_size = nombre de mots dans le vocabulaire (ici 1000). C'est la taille du dictionnaire.

embedding_dim = taille des vecteurs qui représentent chaque mot (ici 16). C'est la "richesse" de la représentation.

4. En quoi consiste une couche Embedding ?

C'est une table de recherche : elle associe à chaque indice de mot un vecteur de dimension embedding_dim. Ces vecteurs sont appris pendant l'entraînement. On la place avant le RNN car le RNN a besoin de vecteurs denses, pas d'indices bruts.

5. Quelle est la différence entre SimpleRNN et LSTM ?

SimpleRNN : mémoire courte, souffre du vanishing gradient sur les longues séquences.

LSTM : utilise des portes (input, forget, output) pour contrôler l'information. Mémoire plus longue, plus adapté aux longues séquences.

On préfère un LSTM quand la séquence est longue et que le contexte lointain est important.

6. Que représente window_size=3 ?
C'est le nombre de mots précédents utilisés pour prédire le mot suivant. Avec 3, on regarde les 3 derniers mots. Plus il est grand, plus le contexte est riche, mais plus le modèle est complexe.

7. Pourquoi sparse_categorical_crossentropy ?
Parce que les cibles y sont des indices de mots (entiers), pas des vecteurs one-hot. sparse_categorical_crossentropy gère directement les entiers.

8. Que fait EarlyStopping ?
Il arrête l'entraînement quand la loss ne baisse plus pendant patience époques (ici 10). Avec restore_best_weights=True, il restaure les meilleurs poids. Ça évite le sur-apprentissage et fait gagner du temps.

9. Comment savoir si le modèle converge bien ?
En regardant la courbe de loss : elle doit baisser puis se stabiliser. Si elle remonte, c'est du sur-apprentissage. Si elle stagne trop tôt, le modèle n'apprend pas assez.

10. Analyse le texte généré
Le texte généré est souvent grammaticalement incorrect et incohérent. Pourquoi ?

Le corpus est petit (quelques poèmes)

Le modèle est simple (SimpleRNN 32 neurones)

window_size=3 est court (contexte limité)

Le vocabulaire est limité à 1000 mots

Il n'y a pas de compréhension du sens, juste des statistiques de succession de mots

Pour écrire comme Hugo, il faudrait un grand modèle de langage (GPT), entraîné sur des milliards de mots.